Advanced Machine Learning

DATA 642

Lecture 6 — Introduction to PCA

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

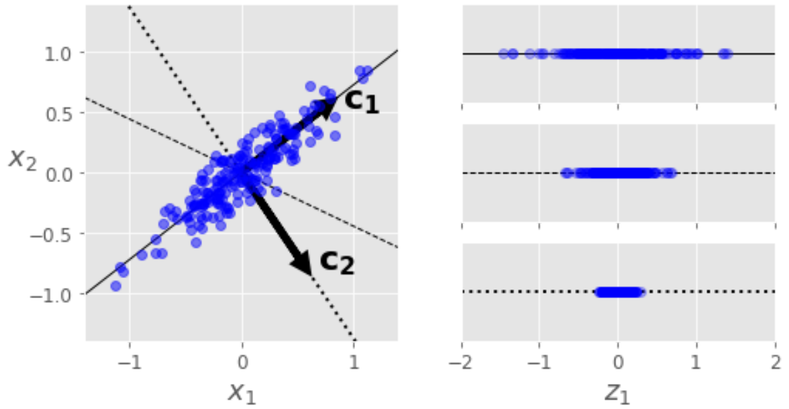
Principal Component Analysis (PCA)

PCA is by far the most popular dimensionality reduction algorithm. It identifies the hyperplane that maximizes the variance of the data and then projects the data onto it, effectively reducing the dimensionality while preserving the most important features.

- PCA was first introduced by Karl Pearson in 1901 as "the principal axis theorem" in the context of his work on the statistical analysis of biological data.
- However, the modern formulation and name "Principal Component Analysis" were popularized by Harold Hotelling in the 1930s.
- Initially, PCA was developed within the field of statistics to address problems related to data analysis and dimensionality reduction.

Preserving the Variance

- Choosing the right hyperplane is crucial before projecting the training set onto a lower-dimensional space.
- For example, a simple 2D dataset is represented on the left of the following figure, along with three different axes (i.e., one-dimensional hyperplanes). On the right is the result of the projection of the dataset onto each of these axes.
- The solid line preserves maximum variance, while the dotted line captures very little, and the dashed line maintains an intermediate amount.
- Selecting the axis that preserves maximum variance minimizes information loss, a fundamental concept driving PCA.



Principal Components

- The assumption underlying PCA, as well as any dimensionality reduction technique, is that the observed data are generated by a system or process that is driven by a (relatively) small number of latent (not directly observed) variables. The goal is to learn this latent structure.

- PCA identifies the axis that accounts for the largest amount of variance in the training set. In the above figure, it is the solid line. It also finds a second axis, orthogonal to the first one, that accounts for the largest amount of remaining variance.

- If it were a higher-dimensional dataset, PCA would also find a third axis, orthogonal to both previous axes, and a fourth, a fifth, and so on—as many axes as the number of dimensions in the dataset.

- The direction of the principal components is not stable: if you perturb the training set slightly and run PCA again, some of the new PCs may point in the opposite direction of the original PCs. However, they will generally still lie on the same axes. In some cases, a pair of PCs may even rotate or swap, but the plane they define will generally remain the same.

Maximum variance perspective

Given a set of observation vectors, $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$, which will be assumed to be of zero mean (otherwise the mean/sample mean is subtracted), PCA determines a subspace of dimension $m \leq l$, such that after projection on this subspace, the statistical variation of the data is optimally retained. This subspace is defined in terms of m mutually orthogonal axes known as principal axes or principal directions, which are computed so that the variance of the data, after projection on the subspace, is maximized.

We will derive the principal axes in a step-wise fashion. First, assume that $m = 1$ and the goal is to find a single direction in $\mathbb{R}^l$ so that the variance of the corresponding projections of the data points is maximized. Let $\boldsymbol{u}_1$ denote the principal axis. The variance of the projections (having assumed centered data) is given by

$$J(\boldsymbol{u}_1) = \boldsymbol{u}_1^\top \mathbf{S} \boldsymbol{u}_1, \tag{1}$$

where $\mathbf{S} = \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{x}_n \boldsymbol{x}_n^\top = \frac{1}{N} \mathbf{X}\mathbf{X}^\top$ is the biased version of the empirical data covariance matrix.

The task now becomes that of maximizing the variance. However, because we are only interested in directions, the principal axis will be represented by the respective unit norm vector. Thus, the optimization task is cast as

$$\begin{aligned} \boldsymbol{u}_1 = \underset{\boldsymbol{u}}{\operatorname{argmax}} \quad & \boldsymbol{u}^\top \mathbf{S} \boldsymbol{u} \\ \text{subject to} \quad & \boldsymbol{u}^\top \boldsymbol{u} = 1 \end{aligned} \tag{2}$$

This is a constrained optimization problem and the corresponding Lagrangian is given by

$$\mathcal{L}(\boldsymbol{u}, \lambda) = \boldsymbol{u}^\top \mathbf{S} \boldsymbol{u} - \lambda(\boldsymbol{u}^\top \boldsymbol{u} - 1). \tag{3}$$

Taking the gradient and setting it equal to zero we get

$$\mathbf{S} \boldsymbol{u} = \lambda \boldsymbol{u}. \tag{4}$$

In other words, the principal direction is an eigenvector of the empirical data covariance matrix. If we plug $\mathbf{S} \boldsymbol{u} = \lambda \boldsymbol{u}$ into $\boldsymbol{u}^\top \mathbf{S} \boldsymbol{u}$ and taking into account that $\boldsymbol{u}^\top \boldsymbol{u} = 1$ we obtain

$$\boldsymbol{u}^\top \mathbf{S} \boldsymbol{u} = \lambda. \tag{5}$$

- Hence, the variance is maximized if $\boldsymbol{u}_1$ is the eigenvector that corresponds to the maximum eigenvalue λ_1. Recall that because the (sample) covariance matrix is symmetric and positive semidefinite, all the eigenvalues are real and nonnegative. Assuming $\mathbf{S}$ to be invertible (hence, necessarily, $N > l$), the eigenvalues are all positive, that is, $\lambda_1 > \lambda_2 > \cdots > \lambda_l > 0$, and we also assume they are distinct, in order to simplify the discussion.

- The second principal component is selected so that it (a) is orthogonal to $\boldsymbol{u}_1$ and (b) maximizes the variance after projecting the data onto this direction. Following similar arguments as before, a similar optimization task results with an extra constraint, $\boldsymbol{u}^\top \boldsymbol{u} = 0$. It can easily be shown that the second principal axis is the eigenvector corresponding to the second-largest eigenvalue, λ_2. The process continues until m principal axes have been obtained; they are the eigenvectors corresponding to the m largest eigenvalues.

Practical Aspects

- Computing eigenvalues and eigenvectors is also important in other fundamental machine learning methods that require matrix decompositions.

- We can solve for the eigenvalues as roots of the characteristic polynomial. However, for matrices larger than 4×4 this is not possible because we would need to find the roots of a polynomial of degree 5 or higher. However, the Abel-Ruffini theorem (Ruffini1799; Abel, 1826) states that there exists no algebraic solution to this problem for polynomials of degree 5 or more.

- Therefore, in practice, we solve for eigenvalues or singular values using iterative methods, which are implemented in all modern packages for linear algebra.

- In many applications, we only require a few eigenvectors. It would be wasteful to compute the full decomposition, and then discard all eigenvectors with eigenvalues that are beyond the first few. It turns out that if we are interested in only the first few eigenvectors (with the largest eigenvalues), then iterative processes, which directly optimize these eigenvectors, are computationally more efficient than a full eigendecomposition (or SVD).

- In the extreme case of only power iteration needing the first eigenvector, a simple method called the power iteration is very efficient.